

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Claude AI بوت التداول الذكي مع
- يرسل تقارير عند فتح سيشن لندن ونيويورك
- يستجيب لأمر "بدء" بتحليل عميق 12 نقطة
- يركز على: EUR/USD, GBP/USD, BTC
"""

import urllib.request
import urllib.error
import json
import time
import logging
import sys
import threading
import xml.etree.ElementTree as ET
from datetime import datetime
from zoneinfo import ZoneInfo
from pathlib import Path

# CONFIG
BOT_TOKEN      = "8286666952:AAHiQb-p8Lkkg58CD6Huf5OerkdH4H_BzkI"
CHAT_ID        = "6097396379"
ANTHROPIC_KEY  = "sk-ant-api03-g0uQAdarXXoIi1F16Ih2t2cgp4wxv9TzKN0G1qb0G6VQjKD6qZ3

NY_TZ  = ZoneInfo("America/New_York")
MOR_TZ = ZoneInfo("Africa/Casablanca")

LONDON_HOUR  = 4
NY_HOUR      = 9
POLL_INTERVAL = 30
NEWS_INTERVAL = 15 * 60

CACHE_FILE  = Path.home() / ".smart_bot_seen.json"
LOG_FILE    = Path.home() / "smart_bot.log"
OFFSET_FILE = Path.home() / ".smart_bot_offset.json"

RSS_FEEDS = [
    {"name": "ForexLive",      "url": "https://www.forexlive.com/feed/news"},
    {"name": "Investing.com",  "url": "https://www.investing.com/rss/news.rss"},
    {"name": "CoinDesk",       "url": "https://www.coindesk.com/arc/outboundfeeds/r"},
    {"name": "Reuters",        "url": "https://feeds.reuters.com/reuters/businessNe
]

```

```

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler(LOG_FILE, encoding="utf-8"),
        logging.StreamHandler(sys.stdout),
    ],
)
log = logging.getLogger("smart_bot")

# TELEGRAM
def send_telegram(text: str):
    max_len = 4000
    if len(text) > max_len:
        parts = [text[i:i+max_len] for i in range(0, len(text), max_len)]
        for part in parts:
            _send_single(part)
            time.sleep(1)
    else:
        _send_single(text)

def _send_single(text: str):
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
    payload = json.dumps({
        "chat_id": CHAT_ID,
        "text": text,
        "parse_mode": "HTML",
        "disable_web_page_preview": True,
    }).encode("utf-8")
    req = urllib.request.Request(url, data=payload, headers={"Content-Type": "app
try:
    with urllib.request.urlopen(req, timeout=15) as resp:
        body = json.loads(resp.read())
        if not body.get("ok"):
            log.warning("Telegram error: %s", body)
except Exception as e:
    log.error("Telegram failed: %s", e)

def get_updates(offset: int) -> list:
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/getUpdates?offset={offset}&ti
try:
    with urllib.request.urlopen(url, timeout=30) as resp:
        body = json.loads(resp.read())
        if body.get("ok"):
            return body.get("result", [])
except Exception as e:

```

```

        log.error("getUpdates error: %s", e)
    return []

def load_offset() -> int:
    if OFFSET_FILE.exists():
        try:
            return json.loads(OFFSET_FILE.read_text()).get("offset", 0)
        except:
            return 0
    return 0

def save_offset(offset: int):
    OFFSET_FILE.write_text(json.dumps({"offset": offset}))

# RSS
def fetch_rss(feed: dict) -> list:
    import re
    articles = []
    headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}
    try:
        req = urllib.request.Request(feed["url"], headers=headers)
        with urllib.request.urlopen(req, timeout=20) as resp:
            raw = resp.read()
            root = ET.fromstring(raw)
            ns = {"atom": "http://www.w3.org/2005/Atom"}
            items = root.findall("./item") or root.findall("./atom:entry", ns)
            for item in items[:10]:
                def txt(tag, default=""):
                    el = item.find(tag) or item.find(f"atom:{tag}", ns)
                    return (el.text or "").strip() if el is not None else default
                title = txt("title")
                summary = txt("description") or txt("summary") or ""
                link = txt("link")
                if title:
                    articles.append({
                        "source": feed["name"],
                        "title": title,
                        "summary": re.sub(r"<[^>]+>", " ", summary)[:300],
                        "link": link,
                    })
    except Exception as e:
        log.warning("RSS error [%s]: %s", feed["name"], e)
    return articles

def fetch_all_news() -> list:
    all_articles = []
    for feed in RSS_FEEDS:

```

```

        all_articles.extend(fetch_rss(feed))
    return all_articles[:30]

# CLAUDE AI
def ask_claude(prompt: str, max_tokens: int = 3000) -> str:
    url = "https://api.anthropic.com/v1/messages"
    payload = json.dumps({
        "model": "claude-sonnet-4-5",
        "max_tokens": max_tokens,
        "messages": [{"role": "user", "content": prompt}]
    }).encode("utf-8")
    req = urllib.request.Request(url, data=payload, headers={
        "Content-Type": "application/json",
        "x-api-key": ANTHROPIC_KEY,
        "anthropic-version": "2023-06-01"
    }, method="POST")
    try:
        with urllib.request.urlopen(req, timeout=60) as resp:
            body = json.loads(resp.read())
            return body["content"][0]["text"]
    except Exception as e:
        log.error("Claude API error: %s", e)
        return None

# DEEP ANALYSIS (نقطة 12)
def generate_deep_analysis(news_list: list) -> str:
    now_mor = datetime.now(MOR_TZ).strftime("%A %d %B %Y - %H:%M")
    news_text = "\n".join([f"- [{a['source']}] {a['title']}" for a in news_list[:

        prompt = f"""
        أنت محلل ماكرو احترافي متخصص في الفوركس والعملات الرقمية.
        الوقت: {now_mor}
        بتوقيت المغرب | الأصول: EUR/USD, GBP/USD, BTC

        الأخبار الحالية:
        {news_text}

        حلل السوق بعمق باللغة العربية. لكل نقطة أعط: الملاحظة + رأيك الواضح + لماذا

        1) 🌐 الماكرو
        Hawkish/Dovish؟ تحول Fed/ECB/BoE أهم الأخبار الاقتصادية اليوم؟ موقف
        رأيي: [واضح] لأن: [سبب محدد]

        2) 🌐 الجيوسياسية
        Risk-On أم Risk-Off هل هناك توترات جديدة؟ من المستفيد؟
        رأيي: [واضح] لأن: [سبب محدد]

        3) ⚡ الأخبار الحالية
        أبرز أخبار 24 ساعة؟ مفاجئة أم متوقعة؟ كيف تفاعل السوق؟

```

رأيي: [واضح] لأن: [سبب محدد]

4) 🧠 عقلية السوق

؟ أقوى/أضعف عملة؟ المتداولون خائفون أم طامعون؟ Risk-Off أم Risk-On

رأيي: [واضح] لأن: [سبب محدد]

5) 🏢 تفكير المؤسسات

Retail positioning؟ أين تريد المؤسسات السيولة؟ هل ستعكس الجمهور؟ أين

رأيي: [واضح] لأن: [سبب محدد]

6) 📊 ما تسعر السوق

؟ ما لم يتسعر بعد؟ Underpricing أم Overpricing ما الأخبار المسعرة؟

رأيي: [واضح] لأن: [سبب محدد]

7) ⚠️ المخاطر

أحداث مفاجئة محتملة؟ تصريحات مسؤولين؟ السيناريو الأسوأ والأفضل؟

رأيي: [واضح] لأن: [سبب محدد]

8) 🔥 (Intraday) قصير المدى

؟ المحفز اليوم؟ Reversal أم Continuation الاتجاه الحالي؟

رأيي: [واضح] لأن: [سبب محدد]

9) 🧭 (Macro) طويل المدى

؟ العملات القوية هيكلياً؟ Cycle الاتجاه العام؟ أي

رأيي: [واضح] لأن: [سبب محدد]

10) 🎯 قصة السوق

الذي يحرك السوق؟ هل تغير؟ ما الذي يمكن أن يبدله؟ Narrative ما

رأيي: [واضح] لأن: [سبب محدد]

11) ✍️ التفكير العكسي

Pain Trade؟ الفكرة التي الجميع مقتنع بها وقد تكون خاطئة؟ أين

رأيي: [واضح] لأن: [سبب محدد]

12) 💡 منطق الدخول النهائي

شراء/بيع/انتظار - السبب الحقيقي - ما الذي يغير الفكرة؟ 🟢 EUR/USD:

شراء/بيع/انتظار - السبب الحقيقي - ما الذي يغير الفكرة؟ 🟢 GBP/USD:

شراء/بيع/انتظار - السبب الحقيقي - ما الذي يغير الفكرة؟ 🟢 BTC:

"" كن واثقاً ومباشراً. قل "أتوقع" و"أرى". لا تكن محايداً

انتظر 30 ثانية\n...جاري التحليل العميق للسوق") send_telegram("⌚

response = ask_claude(prompt, max_tokens=4000)

if response:

return f""🧠 تحليل السوق العميق - 12 نقطة""

🕒 {now_mor}

```
{response}
```

```
⚡ <i>أدر مخاطرك دائماً</i>""
    else:
        return f""⚠️ Claude AI تعذر الاتصال بـ
📺 أبرز الأخبار:
"" + "\n".join([f"• {a['title'][:80]}" for a in news_list[:5]])

# SESSION REPORT
def generate_session_report(session_name: str, news_list: list) -> str:
    now_mor = datetime.now(MOR_TZ).strftime("%A %d %B %Y - %H:%M")
    session_emoji = "🇬🇧" if "لندن" in session_name else "🇺🇸"
    news_text = "\n".join([f"- [{a['source']}] {a['title']}" for a in news_list[:5]])

    prompt = f""أنت محلل ماكرو احترافي.
الوقت: {now_mor} - سيشن {session_name} | الأصول: EUR/USD, GBP/USD, BTC

الأخبار: {news_text}
```

:اكتب تقرير سيشن مختصر باللغة العربية

- 1) 📅 أهم الأحداث المنتظرة 24-72 ساعة (التوقيت والتوقعات)
- 2) 🌍 مخاطر جيوسياسية غير مسعرة
- 3) 🏦 Fed/ECB/BoE توقعات السوق من
- 4) ⚠️ مخاطر خفية
- 5) 📊 (سبب واضح لكل واحد + 🟢/🔴/🟡) EUR/USD + GBP/USD + BTC :رأبي 🇮🇪

""كن مباشراً وواثقاً

```
    response = ask_claude(prompt, max_tokens=2000)
    if response:
        return f""{session_emoji} <b>تقرير سيشن {session_name}</b>
🕒 {now_mor}
```

```
{response}
```

```
⚡ <i>أدر مخاطرك دائماً</i>""
    else:
        return f""{session_emoji} <b>تقرير سيشن {session_name}</b>
🕒 {now_mor}
📺 "" + "\n".join([f"• {a['title'][:80]}" for a in news_list[:5]])

# BREAKING NEWS
```

```
BREAKING_KEYWORDS = [
    "fed", "fomc", "powell", "rate hike", "rate cut", "inflation", "cpi", "nfp",
    "payrolls", "gdp", "recession", "war", "attack", "sanctions", "opec",
    "bitcoin etf", "btc", "crypto ban", "bank collapse", "emergency",
    "breaking", "flash", "urgent", "ecb", "lagarde", "boe", "bailey",
]
```

```
def is_breaking(article: dict) -> bool:
    text = (article["title"] + " " + article["summary"]).lower()
    return any(kw in text for kw in BREAKING_KEYWORDS)
```

```
# STATE
```

```
class BotState:
```

```
    def __init__(self):
        self.lock = threading.Lock()
        self.seen = self._load_seen()
        self.last_date = ""
        self.london_sent = False
        self.ny_sent = False
        self.cached_news = []
        self.last_news_fetch = 0
```

```
    def _load_seen(self) -> set:
        if CACHE_FILE.exists():
            try:
                return set(json.loads(CACHE_FILE.read_text()).get("seen", []))
            except:
                return set()
        return set()
```

```
    def save_seen(self):
        CACHE_FILE.write_text(json.dumps({"seen": list(self.seen)[-1000:]}))
```

```
    def reset_day(self, date_str: str):
        self.london_sent = False
        self.ny_sent = False
        self.last_date = date_str
```

```
    def get_news(self) -> list:
        now = time.time()
        if now - self.last_news_fetch > NEWS_INTERVAL or not self.cached_news:
            self.cached_news = fetch_all_news()
            self.last_news_fetch = now
        return self.cached_news
```

```
state = BotState()
```

```
# COMMAND HANDLER
def handle_command(text: str):
    text_lower = text.strip().lower()
    trigger_words = ["أخبار", "اخبار", "تحليل", "ابدأ", "بدأ", "بدء", "start", "a"]
    if any(word in text_lower for word in trigger_words):
        log.info("أمر تحليل مستلم")
        news_list = state.get_news()
        analysis = generate_deep_analysis(news_list)
        send_telegram(analysis)
    elif "مساعدة" in text_lower or "help" in text_lower:
        send_telegram("""🤖 <b>أوامر البوت</b>""")
```

🗨️ أخبار أو تحليل أو بدء

← تحليل عميق للسوق (12 نقطة) فوراً

📅 17 تقارير تلقائية

🇬🇧 9:00 ص - تقرير لندن

🇺🇸 2:00 م - تقرير نيويورك

⚡️ ""فوري - عند كل خبر عاجل""

```
# SCHEDULED
```

```
def check_scheduled(news_list: list):
    now_ny = datetime.now(NY_TZ)
    date_str = now_ny.strftime("%Y-%m-%d")
    hour = now_ny.hour
    if state.last_date and state.last_date != date_str:
        state.reset_day(date_str)
    elif not state.last_date:
        state.last_date = date_str
    if hour == LONDON_HOUR and not state.london_sent:
        send_telegram(generate_session_report("🇬🇧 لندن", news_list))
        state.london_sent = True
    if hour == NY_HOUR and not state.ny_sent:
        send_telegram(generate_session_report("🇺🇸 نيويورك", news_list))
        state.ny_sent = True
```

```
# BREAKING
```

```
def check_breaking_news(news_list: list):
    for article in news_list:
        article_id = f"{article['source']}::{article['title'][:50]}"
        if article_id not in state.seen:
            with state.lock:
                state.seen.add(article_id)
            if is_breaking(article):
                now_mor = datetime.now(MOR_TZ).strftime("%H:%M")
                msg = f"⚡️ <b>خبر عاجل</b>\n📰 {article['source']}\n\
                if article.get("link"):
```



```

        msg += f"\n\n🔗 <a href=\"{article['link']}\">اقرأ المزيد</a>"
        send_telegram(msg)
    state.save_seen()

# TELEGRAM LISTENER
def telegram_listener():
    offset = load_offset()
    while True:
        try:
            updates = get_updates(offset)
            for update in updates:
                offset = update["update_id"] + 1
                save_offset(offset)
                message = update.get("message", {})
                text = message.get("text", "")
                chat_id = str(message.get("chat", {}).get("id", ""))
                if text and chat_id == CHAT_ID:
                    log.info("رسالة: %s", text)
                    handle_command(text)
        except Exception as e:
            log.error("Listener error: %s", e)
        time.sleep(POLL_INTERVAL)

# MAIN
def run_bot():
    log.info("البوت يبدأ...")
    send_telegram(
        "🤖 <b>بوت التداول الاحترافي يعمل الآن</b>\n\n"
        "📊 EUR/USD | GBP/USD | BTC\n"
        "🇬🇧 | لندن: 9:00 ص | 🇺🇸 | نيويورك: 2:00 م\n\n"
        "💬 <b>اكتب 'بدء' للحصول على تحليل عميق فوراً</b>"
    )
    listener_thread = threading.Thread(target=telegram_listener, daemon=True)
    listener_thread.start()
    while True:
        try:
            news_list = fetch_all_news()
            state.cached_news = news_list
            state.last_news_fetch = time.time()
            check_scheduled(news_list)
            check_breaking_news(news_list)
        except Exception as e:
            log.error("خطأ: %s", e, exc_info=True)
        time.sleep(NEWS_INTERVAL)

if __name__ == "__main__":
    run_bot()

```

